Flight Price Analysis

How an automated data pipeline turned 57,000 flight bookings into pricing insights for Bangladesh



Project report · November 2026

At a glance

57,000

flight bookings processed automatically, from raw file to final results

100%

of bookings passed every data quality check (none rejected)

+20%

higher average fare in peak travel seasons than in regular periods

+43%

the Hajj season premium, the highest of any season



The key message: the pipeline works end to end, the data is trustworthy, and timing is the biggest driver of what passengers pay. Flying during Hajj, Eid or the winter holidays costs substantially more than flying at other times of year.

What the project delivers



1. Collect

Load the raw booking file into a working (staging) database, making sure nothing is lost on the way.



2. Check

Test every booking for missing, invalid or inconsistent information, and set aside anything that fails.



3. Calculate

Fill in any missing totals and compute four key performance indicators (KPIs) about fares and bookings.



4. Deliver

Store the results in an analytics database, ready for reports, dashboards and further analysis.



Fully automated. A workflow tool called Apache Airflow runs all four stages in order with a single click, and every stage checks its own results before the next one starts.

The data: 15 months of flight bookings

57,000
bookings

24
airlines

8
departure airports in Bangladesh

20
destination airports

15
months: Jan 2025 to Mar 2026

3
travel classes

What each booking records

Airline and route
who flies, from where, to where

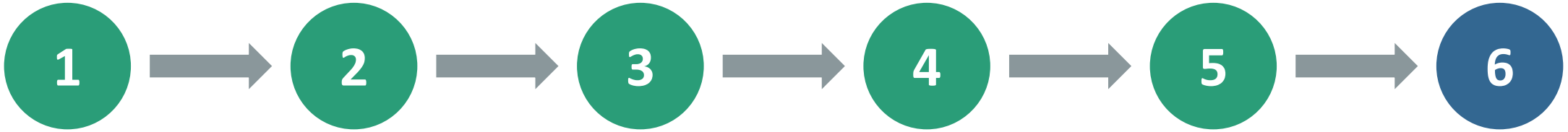
Travel dates
departure, arrival and flight duration

Travel class
Economy, Business or First Class

Fare breakdown
base fare + taxes and surcharges = total fare (in BDT)

Season
Regular, Winter Holidays, Eid or Hajj

Six automated steps, run in order



1 Prepare storage

Creates the database tables if they don't exist yet

`create_tables`

2 Load raw data

Copies all bookings into the staging database and double-checks the count

`ingest_csv`

3 Check quality

Tests every booking against the quality rules; sets aside any that fail

`validate_data`

4 Prepare data

Fills in missing totals; labels peak seasons and routes

`transform_data`

5 Calculate KPIs

Computes the four business measures

`compute_kpis`

6 Deliver

Publishes all results to the analytics database at once

`load_to_postgres`

Each step starts only after the previous one succeeds. If a step fails, it automatically tries again after a minute. If it fails a second time, an alert is raised and the remaining steps wait, so faulty data never reaches the final database.

Three tools, each with one job

Apache Airflow

The conductor

Runs the six steps in the right order, retries failures and raises alerts. Provides a web page to start runs and read logs.

MySQL

The workshop

The staging database: where raw data lands and is checked, cleaned and analysed.

PostgreSQL

The showroom

The analytics database: holds only the final, trusted results, ready for reports and dashboards.

All three run together on one computer inside Docker containers, and start with a single command. A detailed technical diagram is in the project's GitHub README.

Built to be trusted

Checks its own work

Every step counts the data it wrote and stops if the numbers don't add up, so lost or duplicated rows are caught immediately.

Safe to run again

Re-running the pipeline replaces old results instead of adding to them, so a repeat run never double-counts bookings.

All-or-nothing delivery

The final database is updated in one go: either every result table is refreshed, or none is. Readers never see a half-updated report.

Alerts when something breaks

Failed steps retry automatically once; a second failure produces a clear alert naming the step and the error.

Data quality: every booking passed

57,000

of 57,000 bookings passed every check

0

bookings rejected

2,522

kept but flagged for review (4.4%)

What we checked on every booking

- Airline, route, fares and season are all present
- Dates are valid dates
- Airport codes are valid 3-letter codes
- Arrival is after departure
- Fares are real numbers
- Fares are above zero (no negatives)
- Origin and destination differ
- Season is one we recognise

Small fixes made automatically: extra spaces removed and airport codes capitalised. A missing total fare would be calculated rather than rejected.

A puzzle in 2,522 fares

In 4.4% of bookings, the recorded total is higher than base fare + taxes, always by exactly 20%. An example:

Base fare		Taxes		Expected total	VS	Recorded total
10,000 BDT	+	**2,000 BDT**	=	**12,000 BDT**		**14,400 BDT**

14,400 = exactly 1.2 × 12,000

What we did

Kept the recorded fare and added a 'mismatch' flag to these bookings. Nothing was deleted or overwritten.

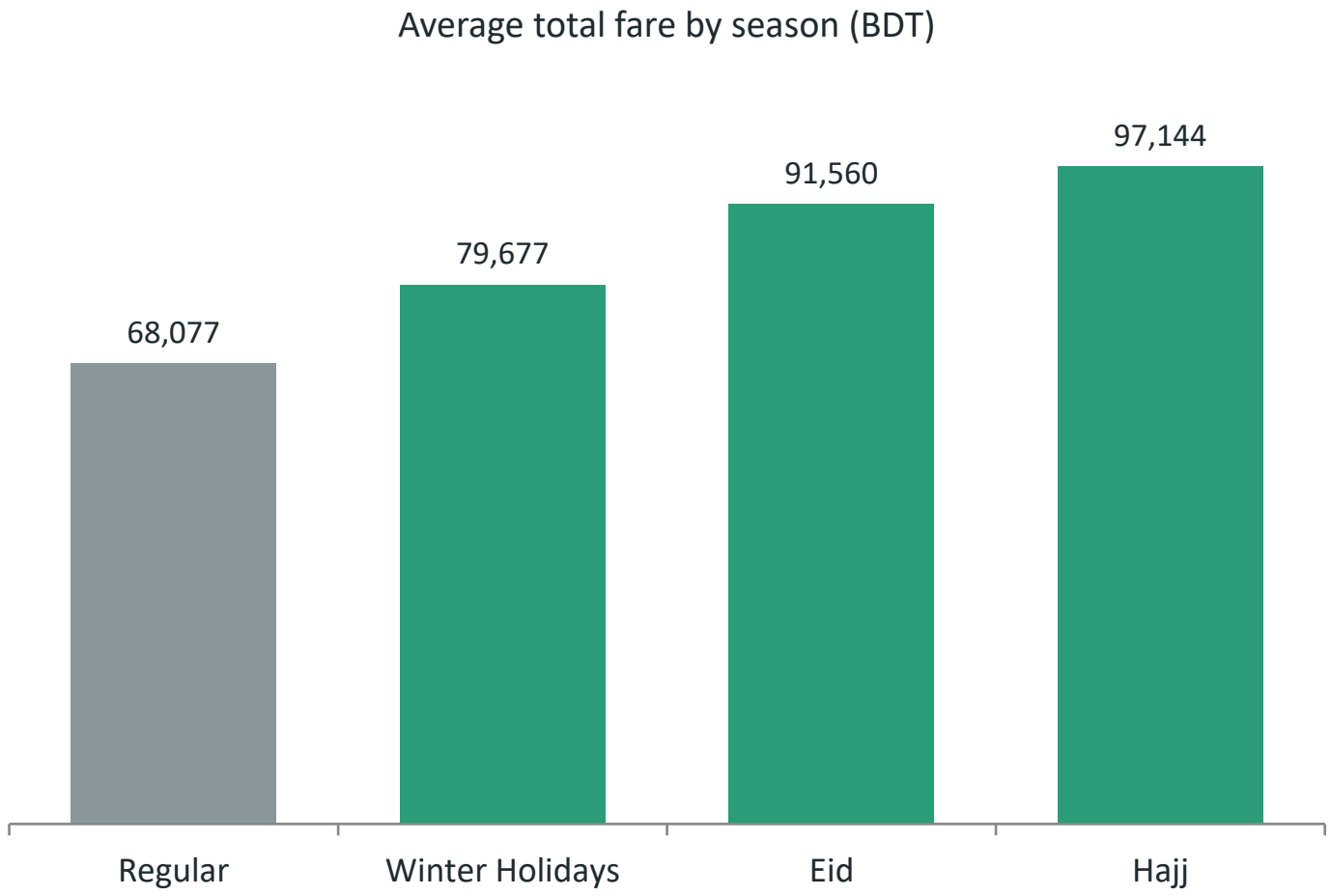
Why

The pattern is too consistent to be a typing error. It most likely reflects a real 20% charge that the dataset doesn't explain.

Recommendation

Ask the data provider what the extra 20% represents, then decide whether to include these bookings in fare averages.

Peak seasons cost about 20% more



+43% **Hajj**
97,144 BDT · 942 bookings

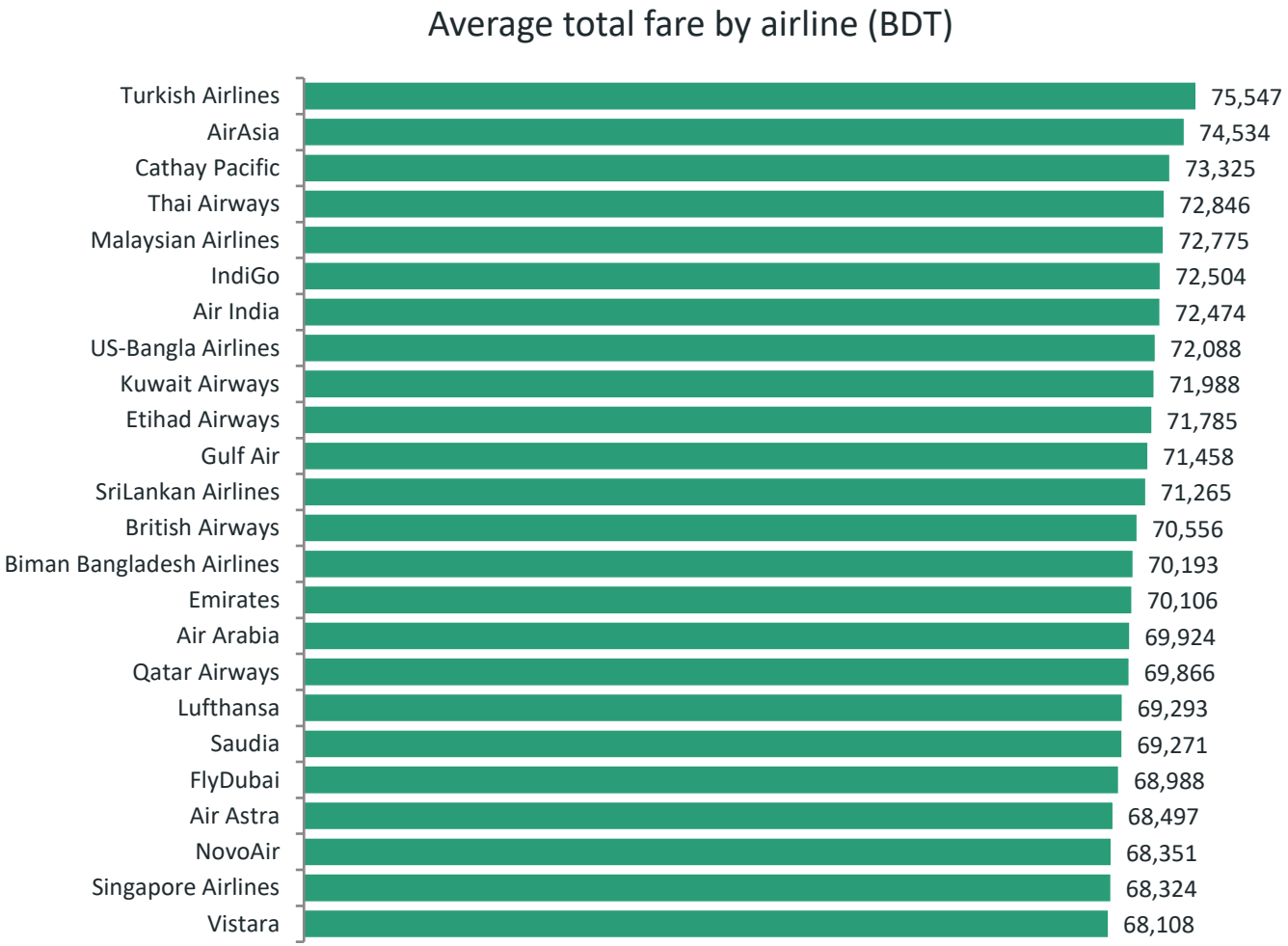
+34% **Eid**
91,560 BDT · 603 bookings

+17% **Winter Holidays**
79,677 BDT · 10,930 bookings

How it's calculated

Average total fare in each season, compared with the regular (non-peak) average of 68,077 BDT. Peak seasons: Eid, Hajj and Winter Holidays.

Airlines charge within a narrow band



Highest: Turkish Airlines
75,547 BDT on average

Lowest: Vistara
68,108 BDT on average

Only 11% separates the most and least expensive airline, far less than the swing between seasons.

How it's calculated

The mean total fare of all bookings for each airline.

US-Bangla leads bookings by a wide margin



4,496

bookings for US-Bangla Airlines (7.9% of all), roughly twice any other airline

The other 23 airlines each have between 2,201 and 2,368 bookings (about 4% each).

How it's calculated

The number of bookings (rows in the data) for each airline, and its share of all bookings.

The most popular routes are nearly tied

Rank	Route	Bookings	Avg fare (BDT)
1	Rajshahi → Singapore	417	113,963
2	Dhaka → Dubai	413	104,942
3	Barisal → Toronto	410	107,844
4	Chittagong → Bangkok	408	107,170
5	Cox's Bazar → Delhi	408	100,161
6	Barisal → Jeddah	407	112,749
7	Chittagong → Cox's Bazar (domestic)	404	7,673

152 routes, each with between 310 and 417 bookings

Domestic routes are popular and cheap. Chittagong → Cox's Bazar ranks 7th at about 7,700 BDT, roughly a fourteenth of a typical international fare.



How it's calculated

Bookings are counted for every origin-destination pair and ranked, most first. Direction matters: Dhaka → Dubai and Dubai → Dhaka are separate routes.

Challenges we solved along the way

Software wouldn't start

Docker wasn't running, settings had typos and two software versions didn't match.

Fix: Fixed each, and wrote step-by-step setup instructions.

Not enough computer memory

The standard setup needs more memory than was available.

Fix: Switched to a lighter setup with fewer background services.

Awkward column names

Names like 'Tax & Surcharge (BDT)' break database code.

Fix: Standardised every name in one central list.

The 20% fare puzzle

2,522 totals didn't match base fare + taxes.

Fix: Kept and flagged them instead of guessing.

Clean data hid untested checks

With no bad rows, the quality checks were never exercised.

Fix: Planted deliberate errors; every one was caught.

Risk of half-updated reports

A failure mid-delivery could leave results partly updated.

Fix: Made delivery all-or-nothing.

A note of caution, and next steps

Read the rankings with care

Bookings are spread unusually evenly across routes and most airlines. Real booking data is rarely this uniform, which suggests the dataset may be partly synthetic.

The seasonal pattern is large and consistent, so it is the most reliable finding. Airline and route rankings should be confirmed with real-world data before acting on them.

The evidence: 152 routes with 310 to 417 bookings each; 23 airlines with about 2,200 to 2,370 each.

Recommended next steps

Explain the 20% markup
Ask the data provider what the extra charge on 2,522 fares represents.

Run on a schedule
Process new booking files automatically as they arrive.

Add a dashboard
Connect a reporting tool to the analytics database for self-service charts.

Email the alerts
Send failure alerts by email instead of only writing them to the log.

Thank you

The full technical documentation, diagrams and code are in the project's GitHub repository (see the README).

